Comparing Optimization Targets for Contrast-Consistent Search

Hugo Fry, Seamus Fallows, Ian Fan, Jamie Wright, Nandi Schoots

The AI Safety Hub

AI Safety Hub

Abstract

We investigate the optimization target of Contrast-Consistent Search (CCS), which aims to recover the internal representations of truth of a large language model. We present a new loss function that we call the Midpoint-Displacement (MD) loss function. We demonstrate that for a certain hyper-parameter value this MD loss function leads to a prober with very similar weights to CCS. We further show that this hyper-parameter is not optimal and that with a better hyper-parameter the MD loss function tentatively attains a higher test accuracy than CCS.

Introducing CCS

Contrast-Consistent Search (CCS), developed by Burns et al. [1], is an unsupervised method for extracting knowledge from the hidden states of large language models. Given only unlabelled model activations, CCS is able to accurately classify statements according to their truth value. It does this by utilising the negation consistency property of truth: a statement and its negation must have opposite truth values. Framed in terms of probabilities, given the probability p that a proposition is true, the probability its negation is true is $1-p$. CCS works by finding a direction in activation space that satisfies this consistency constraint.

Dataset. We take a dataset of *contrast pairs* $\{(x_i^+, x_i^-)\}_{i=1}^n$ consisting of natural language statements x_i^+ and their logical opposites x_i^-. The pairs are formed by taking a question q_i and appending with one of two mutually exclusive answers. Contrast pairs are fed to a pre-trained language model to obtain a set of representations $\{(\phi_i^+, \phi_i^-)\}_{i=1}^n$, where $\phi_i^\pm := \phi(x_i^\pm) \in \mathbb{R}^d$ is the activation vector of a particular layer for the input $x_i^\pm$. These representations are then used to train a linear classifier according to an objective designed to enforce negation consistency on contrast pairs.

In order to avoid training the classifier to simply detect the presence of the mutually exclusive answers, the sets $\{\phi_i^+\}$ and $\{\phi_i^-\}$ should be independently normalized. The normalized representations are given by

$$\tilde{\phi}_i^\pm := \frac{\phi_i^\pm - \mu^\pm}{\sigma^\pm}, \tag{1}$$

where $(\mu^\pm, \sigma^\pm)$ are the means and standard deviations of the respective sets. For convenience, we will omit the tilde and simply use $\phi_i^\pm$ to represent the normalized representations.

Loss Function. A linear classifier $p_{\theta,b} : \phi \to \sigma(\theta^T\phi + b)$ is trained on the normalized activations, where σ is the sigmoid function, θ is a vector of weights and b is a bias. The loss function is given by

$$L_{\text{CCS}}(\theta, b) := \frac{1}{n}\sum_{i=1}^{n} \left[1 - p_{\theta,b}(\phi_i^+) - p_{\theta,b}(\phi_i^-)\right]^2 + \min\left\{p_{\theta,b}(\phi_i^+), p_{\theta,b}(\phi_i^-)\right\}^2. \tag{2}$$

The first term encourages the classifier to find features that are negation-consistent and the second term is included to disincentivise the degenerate assignment $p_{\theta,b}(\phi_i^+) = p_{\theta,b}(\phi_i^-) = 0.5$.

Inference. To make a prediction on an example q_i, after training, the average

$$\bar{p}_i(q_i) := \frac{1}{2}\left(p_{\theta,b}(\phi_i^+) + (1 - p_{\theta,b}(\phi_i^-))\right) \tag{3}$$

is compared to 0.5 with $p_i > 0.5$ corresponding to either the answer "yes" or "no" based on whichever gives the maximum accuracy on a given test set.

Clarifying CCS

A natural guess for how CCS is able to accurately classify truth is that the normalized model representations are approximately clustered according to truth and that CCS is able to find a hyperplane that separates these two clusters. However, this explanation is incorrect.

Recall that after training, an example q_i is classified according to $p(q_i) > 0.5$. Using equation 3, this condition reduces to

$$\sigma(\theta^T\phi_i^+ + b) > \sigma(\theta^T\phi_i^- - b)$$
$$\to \theta^T(\phi_i^+ - \phi_i^-) > 0.$$

We see that CCS is classifying only according to the displacement vectors $(\phi_i^+ - \phi_i^-)$. Since these are translation invariant, CCS does not require the contrast pairs to be separable by a hyperplane. Figure 1 shows an example in which CCS is not finding a separating hyperplane even when it gains a high test accuracy.

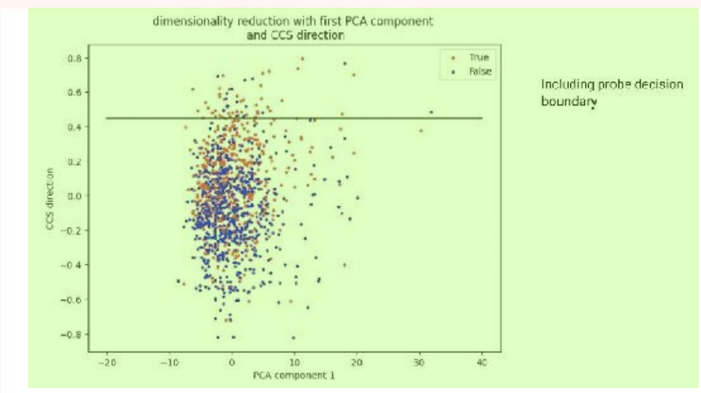

Figure 1. We consider activations of the T5-base model on the BoolQ train dataset. On the x-axis we plot the projections of the activations ϕ_i onto the first principle component and on the y-axis we plot the datapoints ϕ_i projected onto $\hat{\theta}$. We colour the datapoints ϕ_i by the (ground-truth) truth-labels 'true' (red) and 'false' (blue) of the datapoints x_i. The horizontal line indicates the inputs for which CCS outputs 0.5.

Additionally, the original paper [1] presents CCS as learning probabilities for the truth values of contrast pairs. We suggest abandoning this probabilities framing. Figure 2 shows that CCS can still perform well even when the probabilities are strongly clustered around 0.5.

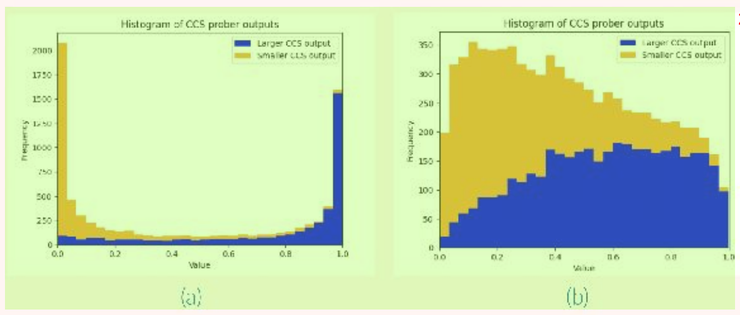

Figure 2. (a) Histogram displaying the CCS prober outputs evaluated on the last hidden state of the encoder of UnifiedQA T5-Large for the BoolQ dataset. (b) Histogram displaying the CCS prober outputs evaluated on the last hidden state of the decoder of UnifiedQA T5-Large for the BoolQ dtatset. Despite the encoder having a higher confidence in the prober outputs, the encoder has a lower test accuracy (0.523) than the decoder (0.978).

Introducing the MD loss function

Using the normalized weight vector $\hat{\theta}$, we define the following quantities:

$$u_i := \phi_i^+ - \phi_i^-, \text{ and}$$
$$\sigma_d^2 := \frac{1}{n}\sum_i (\hat{\theta}^T u_i)^2. \tag{4}$$

Here u_i is the displacement between the activations of a contrast pair and σ_d^2 is the mean square separation of the activations of the contrast pairs along the direction θ.

Furthermore, we analogously define

$$v_i := \phi_i^+ + \phi_i^-, \text{ and}$$
$$\sigma_m^2 := \frac{1}{n}\sum_i (\hat{\theta}^T v_i)^2. \tag{5}$$

Here $\frac{v_i}{2}$ is the midpoint of the activations of a contrast pair and $\frac{\sigma_m^2}{4}$ is the mean square value of the midpoint of the activations of the contrast pairs along the direction θ.

We propose a new loss function and demonstrate that this new loss function is a good proxy optimisation target for CCS. The new loss function is given by

$$L_{\text{MD}} = (\lambda - 1)\sigma_d^2 - \lambda \cdot \sigma_m^2, \tag{6}$$

where $\lambda \in [0, 1]$ is a hyper parameter controlling the relative trade off between σ_d^2 and σ_m^2, and the weight vector θ is constrained to satisfy $|\theta| = 1$.

To understand why we use this loss function, first that the CCS loss function incentivises increasing the separation of the prober outputs of contrast pairs $|p(\phi_i^+) - p(\phi_i^-)|$. Consider a CCS prober $p(\phi) = \sigma(\theta^T\phi + b)$ in which θ is constrained to a fixed norm $|\theta| = c$.

In order for CCS to increase the difference between prober outputs, one might expect that CCS finds a direction that increases the difference of the prober inputs (since sigmoid is a monotonically increasing function). That is to say, one might expect CCS will find a direction that maximises σ_d^2.

However, if σ_m^2 is much larger than σ_d^2, then the input to the sigmoid for each contrast pair (i.e. $\theta^T\phi_i^+ + b$ and $\theta^T\phi_i^- + b$) will be pushed into the same saturation regime of the sigmoid. This results in a lower difference in prober outputs of contrast pairs, which in turn results in a trade off between maximising σ_d^2 while minimising σ_m^2.

It should be stressed that this trade off between σ_d^2 and σ_m^2 is purely an artifact of the double saturation of sigmoid used in the CCS prober. Since this trade off should occur no matter what $|\theta| = c$ is constrained to, we propose that the unconstrained CCS prober is in general optimising for some balance between σ_d^2 and σ_m^2.

Results

The MD-CCS and MD-Accuracy (MD-Acc) loss functions are both trained using the Midpoint-Displacement loss function but with different hyper-parameter searches. For comparison, we include probers trained using a variety of loss functions. The MA and SMR loss functions are both based on taking the mean displacements. The PCA loss function identifies the first principal component of the displacements. The random probers (Rand.) are found by randomly initializing 10 weight vectors and taking the average resulting accuracy. Lastly, the supervised probers (Superv.) are probers trained on labelled data with the same structure as the CCS probers, $p(\phi) = \sigma(\theta^T\phi + b)$.

Similarity

In Table 1 we find that the average cosine similarity between the weight vector of the CCS prober and the weight vector of the prober trained using our new MD method is about 0.63. For reference, the probability of two uniformly sampled 1024-dimensional unit vectors having a cosine similarity of 0.63 or higher is approximately 10^{-237}. Note that the CCS probers had an average cosine similarity with themselves of only 0.78. This suggests that the MD-CCS loss function is a good proxy optimization target for CCS.

Model	Loss Function							
	CCS	MD-CCS	MD-Acc	MA	SMR	PCA	Rand.	Superv.
UQA (E)	0.8359	**0.7034**	0.2995	0.1448	0.1991	0.2406	0.0222	0.2583
UQA (D)	0.8787	**0.7269**	0.5303	0.1687	0.2432	0.1792	0.0228	0.6014
DeBERTa	0.8643	**0.6209**	0.2786	0.2309	0.2024	0.0741	0.0202	0.4617
GPT-Neo	0.5277	**0.4830**	0.4164	0.0226	0.0485	0.1901	0.0245	0.1347
Average	0.7767	**0.6336**	0.3812	0.1418	0.1733	0.1710	0.0224	0.3640

Table 1. We compute the average cosine similarities of the directions found using different loss functions to the directions of 20 CCS probers. We average over five datasets using the activations of four different models. For each row we have emboldened the loss function that obtained the highest average cosine similarity with CCS, not including the CCS loss function. Note that the (E) and (D) refer to the encoder and decoder layers of the UQA model.

Accuracy

In Table 2 we show the test accuracies of probers trained using the MD loss function on various datasets and models. We tentatively find that the accuracies of the new probers are similar to those achieved by CCS, and often out-perform CCS. Note that MD-Acc probers get a higher test accuracy than both the MD-CCS and CCS probers for three out of four models, for an average difference of around 4%.

Model	Loss Function							
	CCS	MD-CCS	MD-Acc	MA	SMR	PCA	Rand.	Superv.
UQA (E)	0.6863	0.6902	0.7414	0.7399	**0.7419**	0.7383	0.6363	0.8839
UQA (D)	**0.8305**	0.8200	0.8180	0.7550	0.7460	0.7525	0.6286	0.9140
DeBERTa	0.7740	0.7855	**0.8735**	0.8650	0.8585	0.8605	0.7288	0.9135
GPT-Neo	0.5510	0.5755	**0.5898**	0.5820	0.5555	0.5737	0.5603	0.7580
Average	0.7105	0.7178	**0.7557**	0.7355	0.7255	0.7313	0.6385	0.8674

Table 2. We compare test accuracies of different loss functions averaged over five datasets, using the activations of a number of models. For each row we have emboldened the loss function that obtained the highest average test accuracy, not including the supervised loss. The (E) and (D) labels refer to the encoder and decoder layers of the UQA model.

References

[1] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering Latent Knowledge in Language Models Without Supervision. December 2022.

hugo.fry@fryfamily.co.uk